

mosync

>Targeting all development platforms out there is getting increasingly complex. What we need is simplification> by Kshitij Sobti

Most app developers end up targeting only the major platforms while neglecting the others. This is precisely where cross-platform SDKs such as MoSync come in. MoSync features an API, and programming language that lets you target nearly every platform out there! Not only does MoSync support the major Android, and iOS platforms, but also supports BlackBerry, Windows Phone (with the newer Metro-style UI), as well as Symbian. Finally it supports the JavaME platform, thereby enabling MoSync applications to be deployed across most feature phones.

To enable web developers to reach all of these platforms, MoSync offers support for using web standards to develop mobile applications. This is in the vein of PhoneGap or Apache Cordova; where you create a web application, which then runs in a web browser embedded inside a native application. The web applications has access to a number of native APIs allowing it to behave for all intents and purposes as an application directly running on the platform. In fact, MoSync is compatible with the PhoneGap / Apache Cordova APIs.

MoSync goes a few steps ahead of Apache Cordova however, as it supports not only pure web-based applications, but also hybrid applications that bridge C++ / C with web code.

Multiple Paths

As we mentioned before, MoSync offers multiple paths to developing applications. Depending on where you are coming from, you have different options. These aren't strict paths though, you have quite a bit of flexibility.

On one side you have native path, using C / C++. If you are porting from desktop to mobile, chances are some of the code in is C / C++, and MoSync will let you reuse that on mobile.

On the other hand, if you already have a web application, and want to bring that to mobile, the HTML path is open to you. MoSync will not only allow packaging and running web apps as native mobile apps, it will also let you use the kind of API that normally a web app would not have access to. For example, your web application can access the camera of the device, the compass, the calendar, the contacts and much more.

A third option is to go for a hybrid application which uses both web and native code. This is very useful if you wish to have a rich UI developed using web standards, while the power-hungry aspects of the application, such as image / audio processing can be handled in native code. Of course you can start by porting your web app to MoSync and incrementally replacing performance-intensive code with C / C++ code. Likewise one could port in the functionality of an existing desktop application as C / C++ code and add an HTML interface on top.

Even among these paths you have multiple options. For example, your C / C++ application could either be pure C / C++ code, or it

could use the NativeUI features that MoSync provides — more on NativeUI later — or it could use OpenGL. Likewise, an HTML application could also use either a web UI or a Native UI.

NativeUI

One of the features that are quickly lost when you consider a cross-platform toolkit is the ability to have a native-looking, native-feeling UI. While cross-platform does mean that you can have the same look and feel for your application on all platforms, this isn't necessarily a benefit.

While keeping the same UI across platforms might make it easier for people to switch from one device to another; it could come at the cost of making it difficult for people to switch from one app to another. While a person is using an app on a phone, they expect it to look and behave in a manner that is consistent with the platform. The minor advantage of offering a consistent UI across platforms is trumped by the major advantage of a consistent UI within the platform.

Many cross-platform toolkits offer a solution for this, and in the case of MoSync, we have NativeUI. NativeUI allows one to use a single API to create applications for Android, iOS and Windows Phone 7.

The NativeUI API features a number of widgets that can be used to create a cross-platform UI. Depending on which platform the application is targeting, the same UI code can look vastly different. MoSync allows you to code based on uniform API, but the end-result is rendered on each platform using their native UI representations.

Let us take a look at a snippet of code from one of the example applications that ships with MoSync, NativeUIDemo:

```
mColorScreen = new ScreenColorList();
mWebScreen = new ScreenWebView();
mImageScreen = new ScreenImageSwiper();
this->addTab(mColorScreen);
this->addTab(mWebScreen);
this->addTab(mImageScreen);
```

Here the ScreenColorList, ScreenWebView, and ScreenImageSwiper classes are the three “tabs” or screens in this application



The same code base, rendered on Android, iOS and Windows Phone